

OpenClaw 科研 Agent 的引用一致性评测

实验架构、流程与工程经验

李佳蓓 马惠文 毛恺诚 陈思琪

2026 年 6 月 5 日

项目目标

核心问题

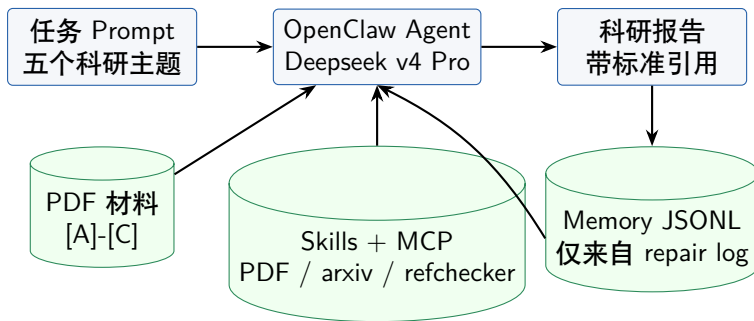
OpenClaw 搭建的科研 Agent 在生成科研报告时，是否会出现“引用与结论不一致”？

这里的“不一致”指：

- 报告中的结论看起来有引用支持；
- 但实际检查引用材料后，发现材料不能充分支持该结论；
- 严重时，引用材料甚至与结论相矛盾。

本项目不修改 Agent 内部代码，而是构造可复现的评测材料、运行脚本、日志和后续标注流程。

被测系统：OpenClaw DeepResearch Agent



固定栈：OpenClaw + Deepseek v4 Pro + PDF skill + citation-standard skill + arxiv MCP + mcp-refchecker + Brave web search。

三段式结构化输出

- ① **总体评估**—基于全部论文概括证据状况、共识/分歧、局限
- ② **逐条结论**—5–8 条可证伪结论，每条带 CPS 出处标注
- ③ **引用论文清单**—完整元信息 + 检索策略

逐条结论格式示例（C2 R1 实际输出）

在纯合成数据替换式训练下，语言模型、扩散模型和 VAE 均必然出现 *model collapse*。
出处：[D] §2.1::¶3

输出 contract：四段 marker

```
# ORIGINAL_REPORT  
→ 阶段 1 初稿（三段式）  
  
# REFCHECKER_REPAIR_LOG  
→ 阶段 2 JSONL 核查记录  
  
# REPAIRED_REPORT  
→ 阶段 3 修复后报告  
  
# RUN_SUMMARY  
→ 运行元数据
```

评测切入点

上述四个 marker 使输出可被程序拆解：
逐条抓取 claim + citation → 进入标注与统计流水线

五个 Case 的设计

Case	主题方向
------	------

- | | |
|----|--|
| C1 | 工具增强 LLM agent 与复杂 coding task |
| C2 | 递归合成数据、弱数据、多模态合成训练、model collapse |
| C3 | 蛋白-配体 docking、分子生成、binding affinity、benchmark leakage |
| C4 | 人类认知 benchmark、多模态推理、cross-scene hallucination |
| C5 | AI coding tools、experienced developer productivity、SWE-Bench Pro |
-

每个 case 固定提供部分文献，并要求 Agent 自主补充/使用相关材料生成报告。不同 case 覆盖不同学科和证据形态，避免只测单一 topic。

引用标准化: citation-standard skill

CPS 语法 (Citation Position Standard)

- 标签: [A] ~ [F] 位置: §4.1::¶2
- 元素: ::paragraph / ::T2 / ::F1 / ::Eq
- 多位置用 ; 分隔

解决的问题

- Abstract → 粒度太粗, 多段落无法定位
- 第 5 页第 3 段 → 计数规则不统一
- limitations 部分 → 自由文本无法解析

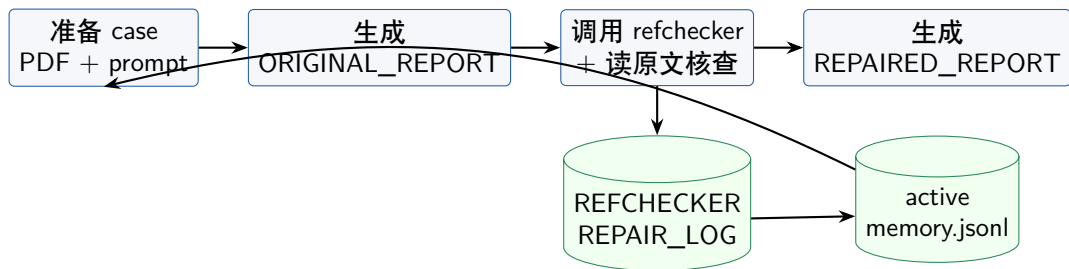
技能组成

- ① SKILL.md — Agent 运行时遵守
- ② cps-spec.md — 完整语法规范
- ③ validate.py — 合规检查脚本

为什么需要

引用不统一 → **人工标注和脚本检查不可复现**。CPS 用封闭词汇表 + 固定语法, 每个 claim-citation pair 可被程序拆解。

整体实验流程



输出 contract 固定为四段：ORIGINAL_REPORT、REFCHECKER_REPAIR_LOG、REPAIRED_REPORT、RUN_SUMMARY。

工具预实验 1: arxiv MCP

目的

测试 Agent 能否自主找到合适的 [D][E][F] 三篇论文，而不是只依赖预置 PDF。

- 条件: `arxiv_off` vs `arxiv_on`
- 规模: $5 \text{ cases} \times 2 \text{ conditions} \times 3 \text{ runs}$
- 每个 run 记录 Agent 选择的三篇论文
- 标注维度: 相关性、全文可得性、重复、是否偏题、是否有效

这个预实验的作用是证明 arxiv MCP 是否值得在后续主实验中固定开启。

arxiv-MCP 贴合分数测试：核心发现

关键发现

arxiv-MCP 开关与论文贴合度并无显著相关性。
大部分开启时贴合得分反而略低于 off。

最值得关注的异常

Case 5 无论开关，贴合程度**显著下降**。

评分方式

LLM as a Judge + 0-4 五档贴合度评分：

- 4 分：主题核心贴合
- 3 分：高度相关，实质贡献
- 2 分：中度相关，偏背景/侧证
- 1 分：弱相关，共享关键词
- 0 分：不相关

校准规则：同领域最多 1-2 分；3-4 分需明确相关且有实质贡献。

条件：5 cases $\times$ 2 conditions $\times$ 3 runs。结论：arxiv-MCP 并非 relevance optimizer。

第一层分析：为什么这个结果本身是合理的

arxiv-MCP 的真实作用

- 优化“能不能找到真实论文”
- 提供更可靠的论文元数据
- 减少幻觉引用 (hallucinated citations)

arxiv-MCP 不做什么

- 不直接优化“和题目有多贴”
- 不是专门的 relevance optimizer
- 不保证每次检索都命中核心文献

核心结论

on 条件更容易找到真实存在、可验证的论文，但这些论文未必更靠近题目中心
工具增强提升的是 **检索真实性与可验证性**，而非主题贴合度上界

每次 run 的输出仍然受检索排序与候选采样的不确定性影响，这是正常的系统行为。

第二层分析：Case 5 的机制深入

C5 arxiv_on 更容易选到的论文类别

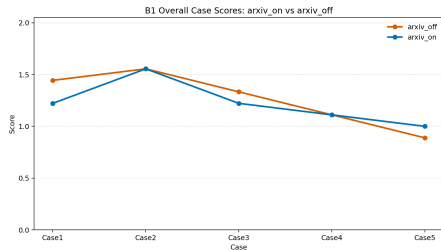
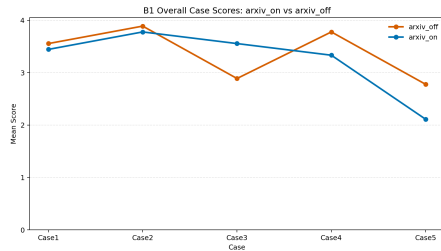
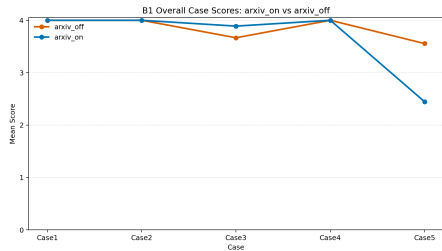
类别	典型论文方向
需求工程	Using LLMs in Software Requirements Specifications
安全风险	Is Vibe Coding Safe?
开发者感知	Will Agents Replace Us?
workflow 影响	Code with Me or for Me?
质量与复杂度	Speed at the Cost of Quality

问题所在

这些论文大多只拿到 1-2 分 (/4)，符合校准规则中“同领域最多 1-2 分”的设定。LLM 判断它们更多谈“软件开发中的 LLM 影响”，而非“替代初级工程师的能力边界”。

这在一定程度上反映了 **LLM 在界定相似度或贴合度方面的局限**。

贴合分数测试：可视化结果



工具预实验 2: refchecker repair

目的

测试 mcp-refchecker 是否能作为生成期工具，帮助 Agent 发现并修复引用元数据和部分引用风险。

- 规模: 5 cases $\times$ 3 runs
- 每次输出两个报告: 原始报告 + repair 后报告
- refchecker 负责核查真实出版物/元数据
- claim support 仍需要 Agent 读原文并做局部判断

限制: refchecker 本身不是“语义支持性判定器”，不能替代人工标注。

主实验：Memory MVP

运行顺序

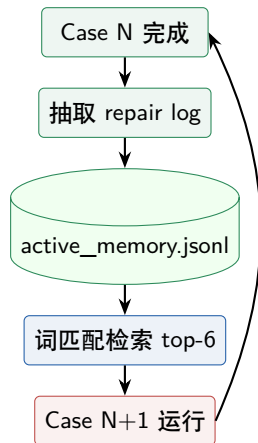
- P1: C1 → C2 → C3 → C4 → C5
- P2: C1 → C2 → C3 → C4 → C5

Memory 来源

- 只写入 repair log
- 不写入完整报告
- 不写入人工标注
- 不作为科学证据引用

研究目标：观察 Agent 是否能从之前的 refchecker/repair 经验中学到更谨慎的引用行为。

Memory 机制：跨 Case 经验迁移



写入 (Write-back)

- 来源：REFCHECKER_REPAIR_LOG 的 JSONL 行
- 包括 clean rows 与 error rows，按 memory_record_id 去重
- 规范化为摘要、关键词、错误类型、修复动作
- **不写入**：完整报告 / 人工标注

检索 + 注入

- TF-IDF-like 词匹配检索 **top-6**
- 注入为 MEMORY_CONTEXT procedural caution
- **不作为科学证据 不可被引用**

Memory 实现细节：实际代码路径

Run 前：retrieve

- 读取共享文件：active_memory.jsonl
- Query = 当前 case_id + topic + citation/support/refchecker 等关键词
- 默认 top_k=6, context 上限 5000 chars
- 输出 memory_context.md, 再拼到 prompt 前面

Run 后：write-back

- 只有 marker 完整且无 hard failure 才写回
- 抽取 # REFCHECKER_REPAIR_LOG 到 # REPAIRED_REPORT 之间的 JSONL
- 允许类型：reference_metadata / claim_citation_pair
- 当前最终 memory: 97 条 records

分析注意

retrieve_log.jsonl 会保留 retry 和调试历史；正式分析应按最终 P1/P2 的 run.log / manifest 过滤。

当前仓库中保留的主要材料：

- prompt 模板与 filled prompts;
- OpenClaw profile setup scripts;
- preflight audit scripts;
- run manifests、run logs、stderr logs、raw outputs;
- citation-standard validator;
- memory retrieval / writeback scripts;
- 后续标注表与统计脚本占位。

关键原则：报告输出、工具配置、profile、workspace、session key 都要能被追溯。

运行环境与工具链问题

网络与检索层

- **arXiv DNS / 代理异常**
Clash Fake-IP 会让 arXiv 解析到异常地址。
- **web search / web fetch timeout**
搜索服务与 arXiv 访问路径不同，代理策略必须分开。

处理

固定 HTTPS_PROXY / HTTP_PROXY，并用 NO_PROXY 让 arxiv.org 与 export.arxiv.org 直连。

MCP 与插件层

- **arxiv MCP -32000**
MCP server 手动握手正常，但 OpenClaw bundle-MCP 层偶发连接失败。
- **profile 配置漂移**
DeepSeek auth、Brave provider、memory/session search 可能在新 profile 中不一致。

处理

增加 safe wrapper、profile audit 和 manifest；统一 Brave provider，显式关闭非实验变量工具。

工具链问题如何约束实验设计

工程风险	若不控制会怎样	实验中的控制方式
并行运行不稳定	JOB=2 放大 arXiv、web search、DeepSeek API timeout，失败和模型能力混在一起。	主实验固定 JOB=1；失败 run 保留日志并用新 session key 重试。
workspace / memory 污染	下载论文、临时文件、聊天历史和 memory 写回跨 run 泄漏，破坏变量隔离。	预实验独立 profile / workspace；Memory 只在 M1_memory_on 内按 C1-C5 顺序共享。
输出格式漂移	自由文本引用位置无法稳定解析，claim-citation pair 抽取不可复现。	固定四段 marker；开启 citation-standard skill；validator 检查 CPS 格式。
工具能力边界不清	arxiv MCP 或 refchecker 的基础设施问题可能被误判为 Agent 推理能力差异。	先做工具预实验；主实验只解释在固定工具栈下的引用一致性表现。

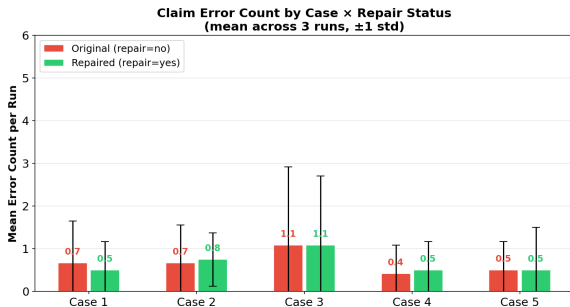
复现边界

prompt + profile audit + run manifest + stderr + raw output + memory snapshot + pass-specific papers。

refchecker Repair 效果：LLM 评测结果

评测方式：citation_check 流水线

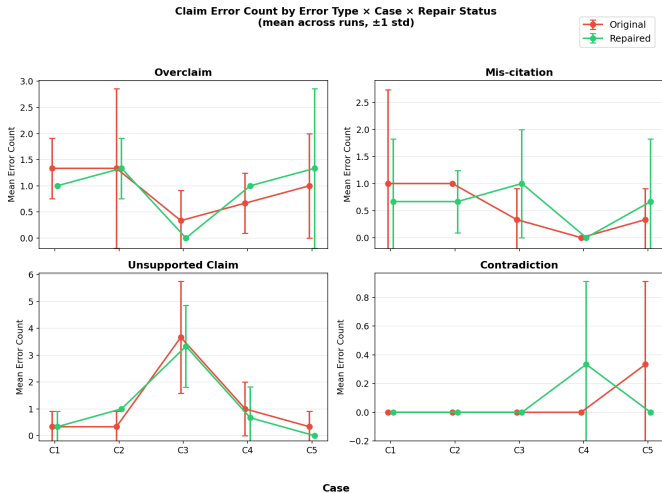
人工定义错误分类体系 + 编写 few-shot prompt $\rightarrow$ build_judge_inputs.py 抽取 claim-citation pair 并构造 batch $\rightarrow$ run_deepseek_judge.py 送入 DeepSeek 批量判定。5 cases $\times$ 3 runs $\times$ repair yes/no, 按 run 取均值 $\pm$ std。



核心结论

- 总错误量：40 $\rightarrow$ 40
repair 没有减少错误总量
- 只有 C1 有改善 (8 $\rightarrow$ 6)
- C2、C4 repair 后反而增加
- C3、C5 完全持平

错误类型分布：2×2 全景



错误类型分布：关键发现

C3 的 Unsupported Claim 是绝对主力 (85%)

- C3 (蛋白-配体 docking) Unsupported Claim 均值 3.3–3.7, 远超其他所有类型
- repair 前后几乎不变: 11 → 10 个
- 这类 claim 元数据完全正确, 但语义支撑不成立

根源: refchecker 的能力边界

refchecker 能做的

验证论文是否存在
核对作者/标题/年份
检查 DOI / arXiv ID

refchecker 不能做的

判断结论是否被引用支撑
评估专业领域的证据强度
发现“听起来科学但无定量支撑”的
claim

Citation verification $\neq$ **Claim verification** —— 领域越专业, 语义支撑判断越需要领域 expertise, refchecker 越无能为力。

Case A: Mis-citation — 引用沾边但接不上

- ① **C1 (SWE-Bench)**: “精心设计的 agent-computer interface 是将 LLM 转化为高效编码 agent 的关键，直接使用 vanilla shell 交互会导致性能大幅下降。”
→ 被引论文主题相关，但讨论的是另一个对象/任务，引用位置无法支撑该具体论断。
- ② **C2 (Model Collapse)**: “仅在合成数据上递归训练的统计语言模型会发生完全塌缩，学到的条件分布以概率 1 收敛至 Dirac 分布。”
→ 沾边但实际讨论条件不同，属于“听起来像但实际不是”。
- ③ **C5 (AI Coding)**: “开发者编码经验不预测 AI 工具采纳率，但显著影响其对 AI 角色的心理模型。”
→ 引用定位错误，被引论文的结论范围与此 claim 不匹配。

Failure Case 实例（续）

Case B: Overclaim — 结论强于原文证据

- ❶ C1: “工具增强型 LLM agent 的 Pass@1 可从 17% 提升至 53%。” → 把单次实验数值夸大为普遍结论，忽略实验设定局限。
- ❷ C4: “MLLM 在透视感知中存在系统性缺陷，与人类差距显著。” → 将有限 benchmark 的观测推广为“系统性缺陷”。
- ❸ C5: “AI 更适合辅助而非替代有经验的工程师。” → 原文仅有用户反馈分析，并未设计对照实验支撑强因果结论。

Case B: Contradiction — 罕见但最严重

- ❶ C5: “使用 AI 后任务时间延长约 32%” — 原文部分数据指向效率提升，方向相反。
- ❷ C4: “异构物体组合引发更严重幻觉，GPT-4o 也不例外” — 被引论文结论方向与该表述冲突。

Repair 的本质：错误类型漂移，而非消除

20 个配对 claim 中，7 个在 repair 后错误类型改变

变化方向	次数	是好是坏？	含义
Overclaim → Mis-citation	2	不确定	措辞弱化了，但引用位置错了
Mis-citation → Overclaim	2	变差	引用修对了，但表述夸大了
Mis-citation → Unsupported Claim	1	暴露问题	引用修对后，证据不足被暴露
Contradiction → Overclaim	1	不确定	矛盾消除了，但变成了夸大
Unsupported Claim → Mis-citation	1	不确定	加了引用，但引错了

Agent 基于 refchecker 反馈改写时，缺乏对原文证据的深入理解

改写方向是**随机的**——有时变好，有时变差，有时只是换了一种错法。Repair 的 prompt 目标应该是**消除错误**，而非仅仅是改写。

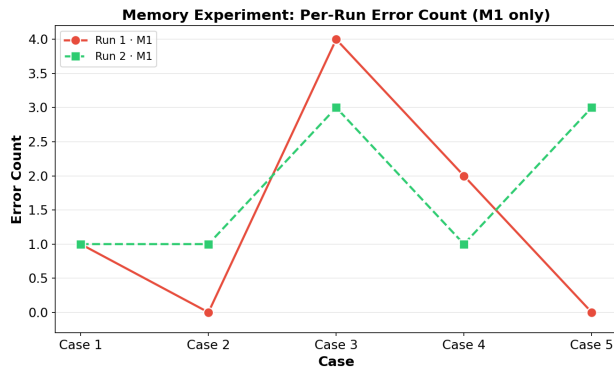
归因分析总结

- ① **refchecker 解决的是引用真实性，不是引用支撑性**
→ C3 Unsupported Claim repair 前后: 11 → 10, 几乎无效
- ② **repair 效果与 Agent 对领域的先验知识正相关**
→ 只有 C1 (最主流 CS topic) 有改善
- ③ **repair 是错误类型漂移，不是错误消除**
→ 7/20 claim 错误类型改变，总错误量不变 (40→40)
- ④ **领域专业度越高，repair 越无效**
→ C3 (生化) 完全无效，C1 (CS) 轻微有效
- ⑤ **Unsupported Claim 是当前系统最致命的盲区**
→ 占总错误 46%，且 repair 完全无法解决

改进方向

面向 source-grounded check 升级；高专业度 case 采用更保守的 claim 策略；repair 改为“改写 → 再核查 → 再改写”迭代循环。

Memory 实验： M1 错误总数（per run）



实验设置

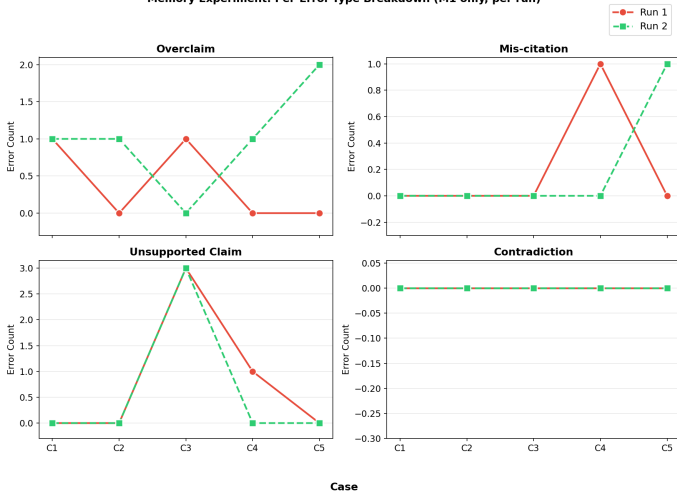
- M1 = memory on（只注入 repair log）
- 5 cases × 2 runs
- LLM 逐 claim 错误分类

关键发现

- C1 错误数最低 (R1=1, R2=1)
- C3 最高 (R1=4, R2=3), 两 run 一致
- C5 两 run 差异大 (0 vs 3)

Memory 实验：分错误类型（M1 only）

Memory Experiment: Per-Error-Type Breakdown (M1 only, per run)



Memory 实验：分错误类型分析

Overclaim — 两个 run 分布不一致

- Run1: C1=1, C3=1, 其余为 0 (较干净)
- Run2: C5=2 (最多), C1/C2/C4 各 1
- 两 run 的 C5 差异最大: 0 vs 2

Unsupported Claim — 全集中在 C3

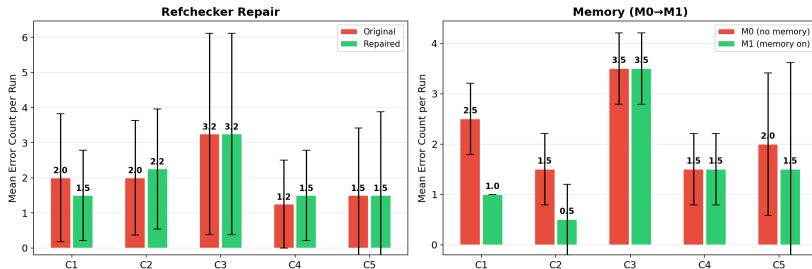
- C3 两 run 各 3 个 (占总 Unsupported Claim 的 100%)
- 蛋白-配体领域的定量 claim 即使有 memory 也依然无法验证语义支撑

好消息

Contradiction 清零—memory 条件下两个 run 都没有出现矛盾型错误。Mis-citation 也极少 (R1: C4=1; R2: C5=1)。

Memory vs Refchecker: 效果对比

Refchecker vs Memory: Error Reduction Comparison



Refchecker Repair

- 总错误 40→40 (无变化)
- 只 C1 轻微改善
- 错误类型漂移, 非消除

Memory (M1)

- 总错误 22→16 (-27%)
- C1 + C2 均改善
- 真正的错误减少

Memory 归因分析

① Memory 比 refchecker repair 更有效

→ Memory 减少的是重复犯错 (Agent 学到之前的教训), refchecker 只做元数据检查

② Memory 帮助最大的是主流 CS 领域 (C1, C2)

→ Agent 能从 SWE-Bench / model collapse 的 repair log 中提取可迁移经验

③ Memory 对 C3 (生化) 完全无效

→ Unsupported Claim 6/6 全在 C3, memory 无法弥补领域知识鸿沟

④ Memory 消除了 Contradiction (最严重错误类型)

→ 说明 Agent 从之前 case 的 repair 中学到了“不要和原文对着干”

⑤ C5 两 run 差异大, 提示 memory 效果不稳定

→ Run1=0, Run2=3, 可能与 C5 自身论文分布广、Agent 检索差异有关

- 科研 Agent 的引用问题不只是“有没有引用”，而是“引用是否真的能支撑结论”；
- 工具链可用性本身需要预实验验证，否则主实验会被基础设施噪声污染；
- citation 格式标准化是后续评测自动化的前提；
- refchecker 对元数据有帮助，但 claim support 仍需要 source-grounded 判断；
- memory 实验必须严格控制顺序和写入来源，否则很容易引入不可解释的混杂。

我们把“科研 Agent 是否会用看似可靠的引用支撑不可靠结论”
转化为一个可复现、可标注、可消融的 OpenClaw 评测流程。

当前重点不是先证明某个错误率，而是先把实验脚手架、工具控制、输出格式和日志链路固定下来。